

طراحی الگوریتم‌ها – الگوریتم‌های حریصانه

Introduction to Algorithm

مهدی جوانمردی

فصل ۱۶ کتاب – الگوریتم‌های حریصانه

Greedy Algorithms 414

- 16.1 An activity-selection problem 415
- 16.2 Elements of the greedy strategy 423
- 16.3 Huffman codes 428

• ۱۶.۱ : مسئله انتخاب فعالیت

• ۱۶.۲ : المان‌های استراتژی حریصانه

• ۱۶.۳ : کد هافمن

Algorithms for optimization problems typically go through a sequence of steps, with a set of choices at each step. For many optimization problems, using dynamic programming to determine the best choices is overkill; simpler, more efficient algorithms will do. A **greedy algorithm** always makes the choice that looks best at the moment. That is, it makes a locally optimal choice in the hope that this choice will lead to a globally optimal solution.

مسئله انتخاب فعالیت

• مجموعه ای از n فعالیت داریم $S = \{a_1, a_2, \dots, a_n\}$

• که هر فعالیت دارای زمان شروع و پایان است:

$$a_i \begin{cases} \text{start time } s_i \\ \text{finish time } f_i \end{cases} \quad 0 \leq s_i < f_i < \infty \quad [s_i, f_i)$$

• فعالیت‌های سازگار:

$$a_i \text{ and } a_j \text{ are compatible} \rightarrow [s_i, f_i) \text{ and } [s_j, f_j) \text{ do not overlap} \begin{cases} s_i \geq f_j \\ \text{or} \\ s_j \geq f_i \end{cases}$$

• هدف: پیدا کردن بزرگترین زیرمجموعه از فعالیت‌های باهم سازگار

• فرض: فعالیت‌های بر حسب زمان پایان مرتب شده اند $f_1 \leq f_2 \leq f_3 \leq \dots \leq f_{n-1} \leq f_n$

$\{a_3, a_9, a_{11}\}$

$\{a_1, a_4, a_8, a_{11}\}$

$\{a_2, a_4, a_9, a_{11}\}$

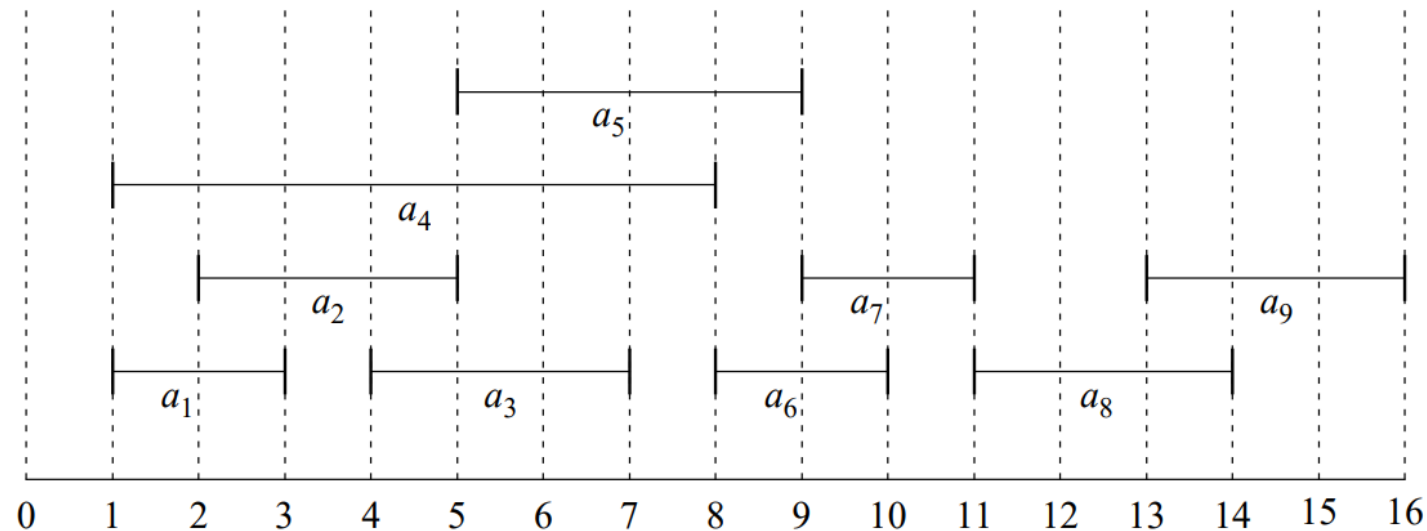
i	1	2	3	4	5	6	7	8	9	10	11
s_i	1	3	0	5	3	5	6	8	8	2	12
f_i	4	5	6	7	9	9	10	11	12	14	16

مثال:

مسئله انتخاب فعالیت

S sorted by finish time: *[Leave on board]*

i	1	2	3	4	5	6	7	8	9
s_i	1	2	4	1	5	8	9	11	13
f_i	3	5	7	8	9	10	11	14	16

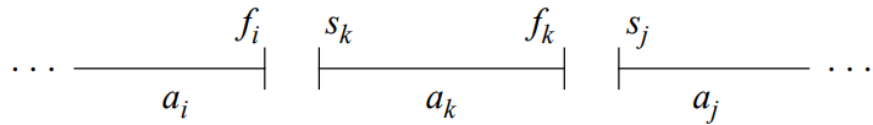


Maximum-size mutually compatible set: $\{a_1, a_3, a_6, a_8\}$.

Not unique: also $\{a_2, a_5, a_7, a_9\}$.

راه حل برنامه نویسی پویا مسئله انتخاب فعالیت

• زیرساختار بهینه S_{ij} = $\{a_k \in S : f_i \leq s_k < f_k \leq s_j\}$ set of activities that start after activity a_i finishes and that finish before activity a_j starts



find a maximum set of mutually compatible activities in S_{ij}

A_{ij}

includes some activity a_k

$S_{ik} \quad a_k \quad S_{kj}$

$$A_{ik} = A_{ij} \cap S_{ik} \quad A_{kj} = A_{ij} \cap S_{kj}$$

$$A_{ij} = A_{ik} \cup \{a_k\} \cup A_{kj}$$

$$|A_{ij}| = |A_{ik}| + |A_{kj}| + 1$$

the size of an optimal solution

$c[i, j]$

$$c[i, j] = c[i, k] + c[k, j] + 1$$

$$c[i, j] = \begin{cases} 0 & \text{if } S_{ij} = \emptyset \\ \max_{a_k \in S_{ij}} \{c[i, k] + c[k, j] + 1\} & \text{if } S_{ij} \neq \emptyset \end{cases}$$

examine all activities in S_{ij}

(16.2)

انتخاب حریصانه!

- چی می‌شد اگر بدون حل همه انتخاب‌های ممکن فقط با یک انتخاب به جواب می‌رسیدیم؟
- انتخاب شهودی و حریصانه:
- انتخاب فعالیتی که منابع را برای فعالیت‌های بیشتری آزاد می‌گذارد
- یعنی انتخاب فعالیتی در S که زودترین زمان پایان را دارد!

i	1	2	3	4	5	6	7	8	9	10	11
s_i	1	3	0	5	3	5	6	8	8	2	12
f_i	4	5	6	7	9	9	10	11	12	14	16

- انتخاب حریصانه در این حالت یعنی انتخاب a_1
- بعد از انتخاب حریصانه فقط فعالیت‌های باقی می‌ماند که بعد از پایان آن شروع شوند

$$S_k = \{a_i \in S : s_i \geq f_k\} \quad \text{the set of activities that start after activity } a_k \text{ finishes}$$

اثبات راه حل حریصانه برای مسئله انتخاب

قضیه:

- زیرمسئله S_k را در نظر بگیرید و فرض کنید a_m فعالیتی در آن با سریع‌ترین زمان پایان باشد
- قطعا a_m در بزرگترین زیرمجموعه باهم سازگار فعالیت‌های مجموعه S_k حاضر خواهد بود

اثبات:

- فرض کنید A_k بزرگترین زیرمجموعه باهم سازگار فعالیت‌های مجموعه S_k باشد
- همچنین a_j فعالیتی در A_k باشد با سریع‌ترین زمان پایان

• فرض: $a_j \neq a_m \longleftarrow f_m \leq f_j$

- در اینصورت فرض کنید $A'_k = A_k - \{a_j\} \cup \{a_m\}$ باشد

- خواهیم داشت که فعالیت‌های A'_k باهم سازگارند و اندازه آن برابر با A_k است

- در نتیجه A'_k نیز یک بزرگترین زیرمجموعه باهم سازگار از S_k است و شامل a_m نیز می‌باشد

راه حل حریصانه بازگشتی مسئله انتخاب فعالیت

i	1	2	3	4	5	6	7	8	9	10	11
s_i	1	3	0	5	3	5	6	8	8	2	12
f_i	4	5	6	7	9	9	10	11	12	14	16

$$S_k = \{a_i \in S : s_i \geq f_k\}$$

RECURSIVE-ACTIVITY-SELECTOR(s, f, k, n)

```

1   $m = k + 1$ 
2  while  $m \leq n$  and  $s[m] < f[k]$            // find the first activity in  $S_k$  to finish
3       $m = m + 1$ 
4  if  $m \leq n$ 
5      return  $\{a_m\} \cup \text{RECURSIVE-ACTIVITY-SELECTOR}(s, f, m, n)$ 
6  else return  $\emptyset$ 
```

$$S_0 = S \quad \text{RECURSIVE-ACTIVITY-SELECTOR}(s, f, 0, n)$$

i	1	2	3	4	5	6	7	8	9	10	11
s_i	1	3	0	5	3	5	6	8	8	2	12
f_i	4	5	6	7	9	9	10	11	12	14	16

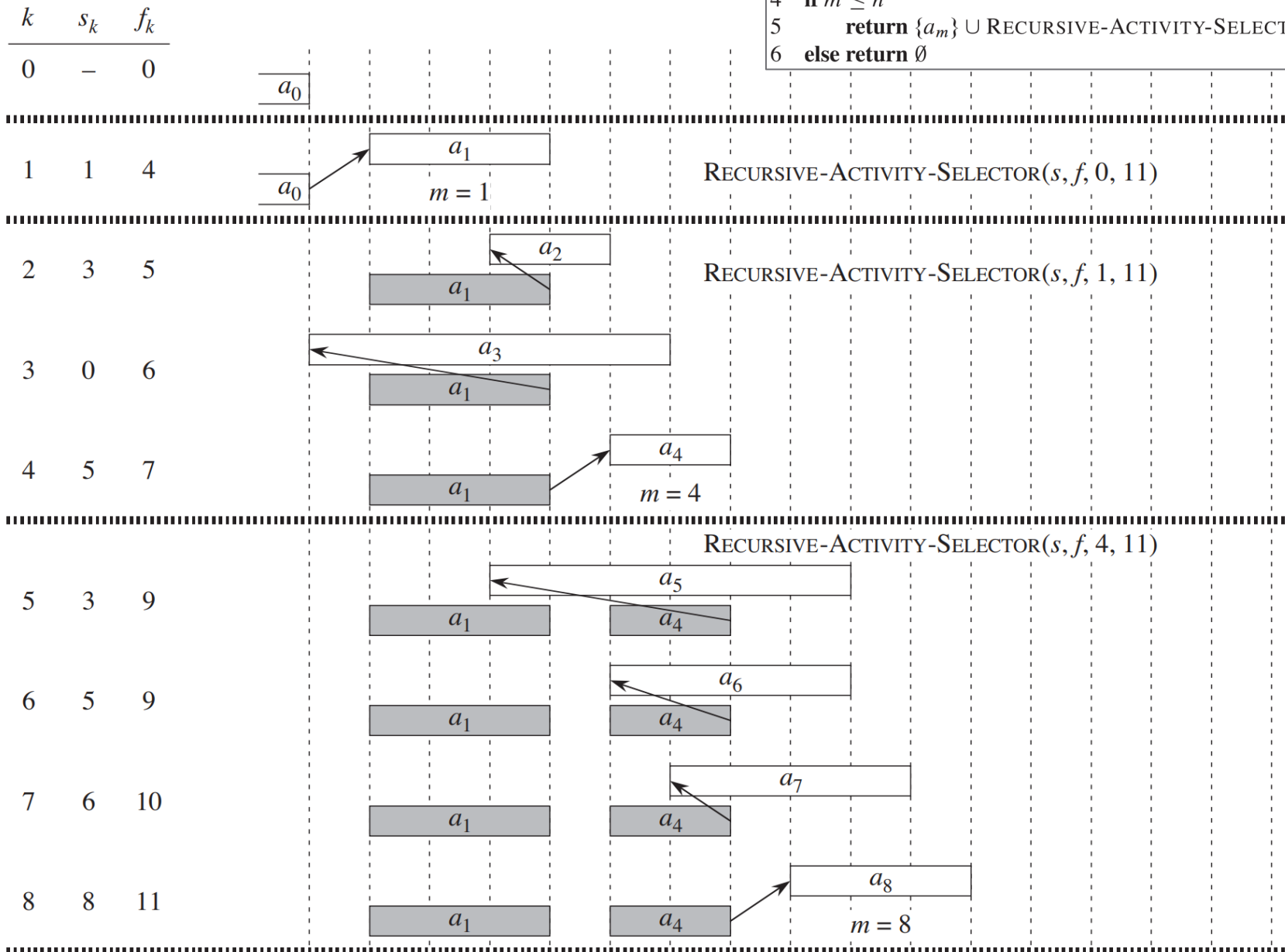
RECURSIVE-ACTIVITY-SELECTOR(s, f, k, n)

```

1   $m = k + 1$ 
2  while  $m \leq n$  and  $s[m] < f[k]$       // find the first activity in  $S_k$  to finish
3       $m = m + 1$ 
4  if  $m \leq n$ 
5      return  $\{a_m\} \cup \text{RECURSIVE-ACTIVITY-SELECTOR}(s, f, m, n)$ 
6  else return  $\emptyset$ 

```

مثال



i	1	2	3	4	5	6	7	8	9	10	11
s_i	1	3	0	5	3	5	6	8	8	2	12
f_i	4	5	6	7	9	9	10	11	12	14	16

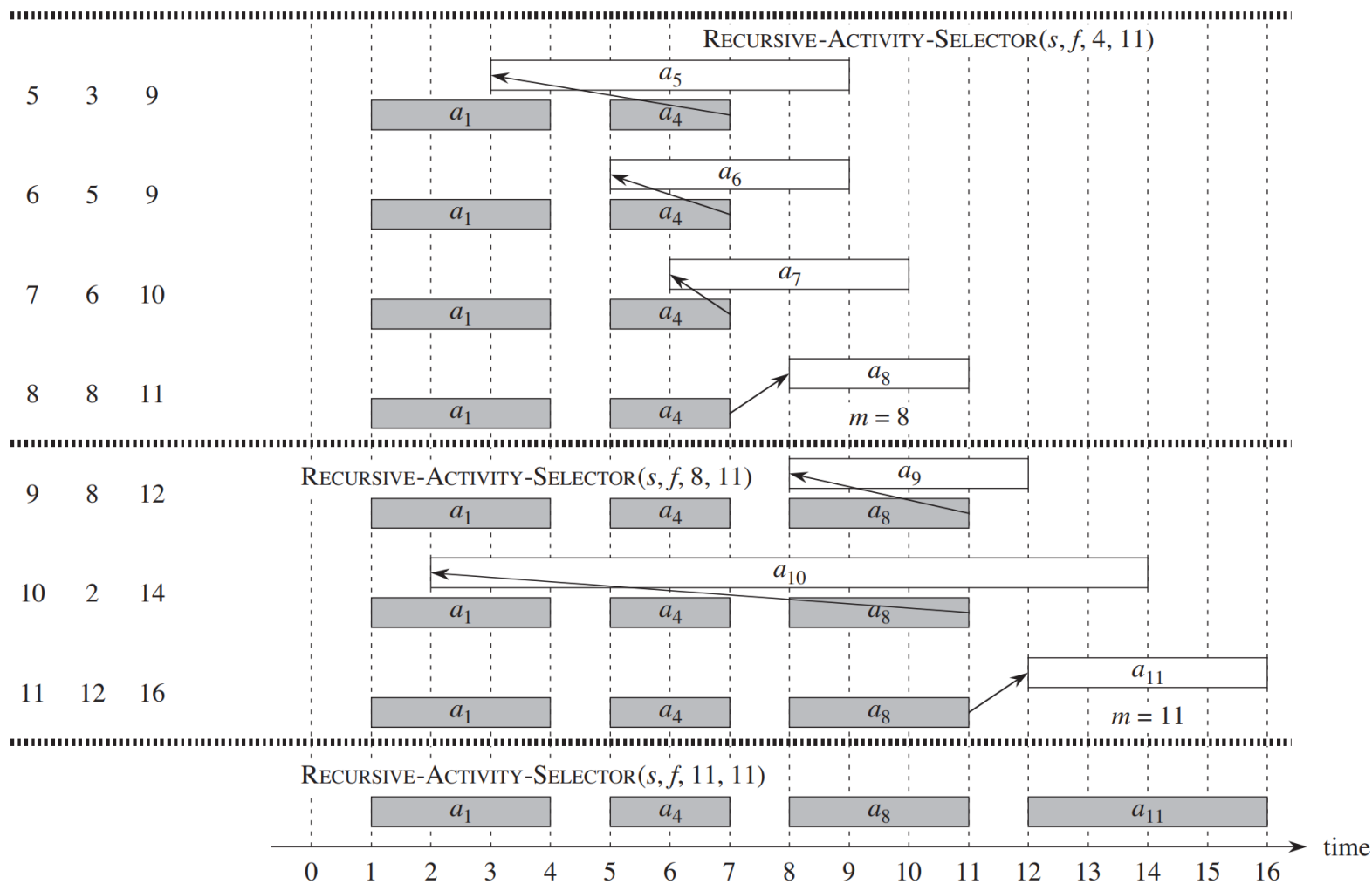
RECURSIVE-ACTIVITY-SELECTOR(s, f, k, n)

```

1   $m = k + 1$ 
2  while  $m \leq n$  and  $s[m] < f[k]$       // find the first activity in  $S_k$  to finish
3       $m = m + 1$ 
4  if  $m \leq n$ 
5      return  $\{a_m\} \cup \text{RECURSIVE-ACTIVITY-SELECTOR}(s, f, m, n)$ 
6  else return  $\emptyset$ 

```

مثال



راه حل حریصانه مستقیم مسئله انتخاب فعالیت

i	1	2	3	4	5	6	7	8	9	10	11
s_i	1	3	0	5	3	5	6	8	8	2	12
f_i	4	5	6	7	9	9	10	11	12	14	16

$$S_k = \{a_i \in S : s_i \geq f_k\}$$

RECURSIVE-ACTIVITY-SELECTOR(s, f, k, n)

```

1   $m = k + 1$ 
2  while  $m \leq n$  and  $s[m] < f[k]$       // find the first activity in  $S_k$  to finish
3       $m = m + 1$ 
4  if  $m \leq n$ 
5      return  $\{a_m\} \cup \text{RECURSIVE-ACTIVITY-SELECTOR}(s, f, m, n)$ 
6  else return  $\emptyset$ 
```

11 $S_0 = S$ RECURSIVE-ACTIVITY-SELECTOR ($s, f, 0, n$)

GREEDY-ACTIVITY-SELECTOR(s, f)

```

1   $n = s.length$ 
2   $A = \{a_1\}$ 
3   $k = 1$ 
4  for  $m = 2$  to  $n$ 
5      if  $s[m] \geq f[k]$ 
6           $A = A \cup \{a_m\}$ 
7           $k = m$ 
8  return  $A$ 
```

المان‌های الگوریتم حریصانه

1. Determine the optimal substructure of the problem.
2. Develop a recursive solution.
3. Show that if we make the greedy choice, then only one subproblem remains.
4. Prove that it is always safe to make the greedy choice. (Steps 3 and 4 can occur in either order.)
5. Develop a recursive algorithm that implements the greedy strategy.
6. Convert the recursive algorithm to an iterative algorithm.

المان‌های الگوریتم حریصانه

1. Cast the optimization problem as one in which we make a choice and are left with one subproblem to solve.
2. Prove that there is always an optimal solution to the original problem that makes the greedy choice, so that the greedy choice is always safe.
3. Demonstrate optimal substructure by showing that, having made the greedy choice, what remains is a subproblem with the property that if we combine an optimal solution to the subproblem with the greedy choice we have made, we arrive at an optimal solution to the original problem.

المان‌های الگوریتم حریصانه

Greedy-choice property:

- Assemble a **globally optimal solution** by making **locally optimal (greedy) choices**

Dynamic programming:

- Make a choice at each step
- But the choice usually depends on the solutions to subproblems
- Solve dynamic-programming problems in **bottom-up manner** (smaller to larger)

Greedy algorithm:

- Make the best choice at the moment and then solve the subproblem that remains
- Greedy choice may depend on choices so far, but not on any future choices
- Make the first choice before solving any subproblems (**top-down manner**)

کد هافمن Huffman Code

• کد هافمن: یک روش برای فشرده سازی داده (معمولا ۲۰ الی ۹۰ درصد)

• دارای جدول نرخ تکرار هر کاراکتر

	a	b	c	d	e	f
Frequency (in thousands)	45	13	12	16	9	5

• ارائه نمایش بهینه برای حرف کاراکتر بصورت یک کد باینری با طول متغیر

• مثال: یک فایل ۱۰۰،۰۰۰ کاراکتری با ۶ کاراکتر مجزا داریم با نرخ تکرار زیر

	a	b	c	d	e	f
Frequency (in thousands)	45	13	12	16	9	5
Fixed-length codeword	000	001	010	011	100	101
Variable-length codeword	0	101	100	111	1101	1100

$$(45 \cdot 1 + 13 \cdot 3 + 12 \cdot 3 + 16 \cdot 3 + 9 \cdot 4 + 5 \cdot 4) \cdot 1,000 = 224,000 \text{ bits}$$

کد هافمن و کد پیشوندی

- کد پیشوندی یا Prefix-code یا بهتر است بگوییم Prefix-free-code:
 - به کدی گفته می شود که هیچ codeword ای در آن پیشوند codeword دیگری نیست
 - حسن کد پیشوندی این است که decode را ساده سازی میکند

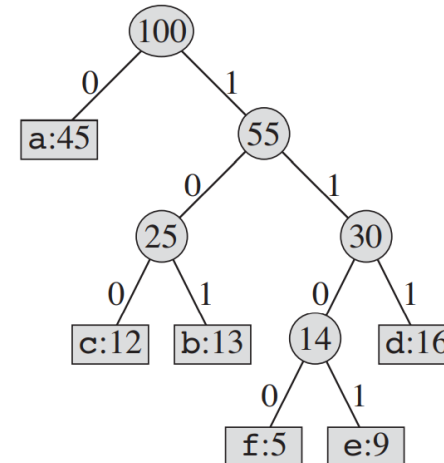
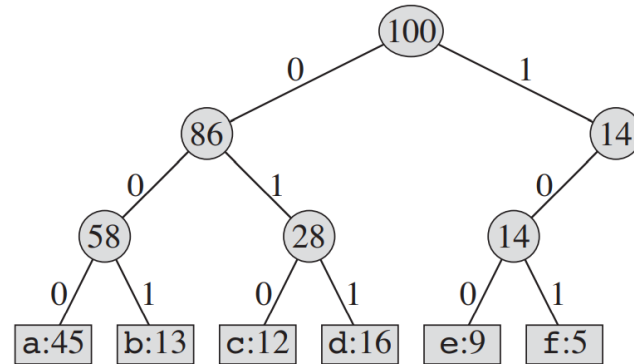
	a	b	c	d	e	f
Frequency (in thousands)	45	13	12	16	9	5
Fixed-length codeword	000	001	010	011	100	101
Variable-length codeword	0	101	100	111	1101	1100

the string 001011101 parses uniquely as $0 \cdot 0 \cdot 101 \cdot 1101$, which decodes to aabe.

نمایش درخت دودویی برای کدگذاری

- درخت دودویی: برگ‌ها معرف کاراکترها ← بازنمایی مناسبی برای کد پیشوندی
- کدورد (codeword) یک کاراکتر ← مسیری که از ریشه تا برگ مربوطه طی شده است
- کدگذاری بهینه برای کل یک فایل همواره در قالب یک درخت دودویی پر (full binary-tree)

	a	b	c	d	e	f
Frequency (in thousands)	45	13	12	16	9	5
Fixed-length codeword	000	001	010	011	100	101
Variable-length codeword	0	101	100	111	1101	1100



نمایش درخت دودویی برای کدگذاری

16.3-2

Prove that a binary tree that is not full cannot correspond to an optimal prefix code.

Let T be a binary tree corresponding to an optimal prefix code and suppose that T is not full. Let node n have a single child x . Let T' be the tree obtained by removing n and replacing it by x . Let m be a leaf node which is a descendant of x . Then we have

$$\text{cost}(T') \leq \sum_{c \in C \setminus \{m\}} c.\text{freq} \cdot d_T(c) + m.\text{freq}(d_T(m) - 1) < \sum_{c \in C} c.\text{freq} \cdot d_T(c) = \text{cost}(T)$$

which contradicts the fact that T was optimal. Therefore every binary tree corresponding to an optimal prefix code is full.

کدگذاری هافمن

- هدف: پیدا کردن کدگذاری بهینه که کمترین تعداد بیت را برای یک فایل استفاده کند
- با فرض داشتن نرخ تکرار هر کاراکتر در کل فایل

$c.freq$ denote the frequency of c

$d_T(c)$ denote the depth of c 's leaf in the tree

$$B(T) = \sum_{c \in C} c.freq \cdot d_T(c)$$

- **آقای هافمن:** یک روش حریصانه برای پیدا کردن بهترین کدگذاری را معرفی کرده است
- روش: پیدا کردن درختی با تعداد n برگ که ویژگی کدگذاری بهینه را داشته باشد
- استفاده از min-priority queue یا صف اولویت مینیمم برای بازگرداندن کاراکترهایی با کمترین میزان تکرار

کدگذاری هافمن

- **آقای هافمن:** یک روش حریصانه برای پیدا کردن بهترین کدگذاری را معرفی کرده است
- روش: پیدا کردن درختی با تعداد n برگ که ویژگی کدگذاری بهینه را داشته باشد

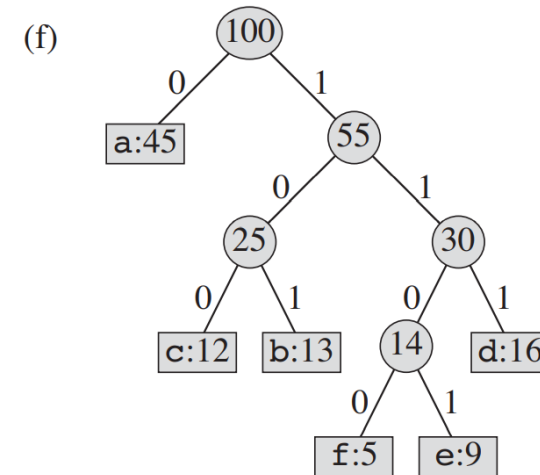
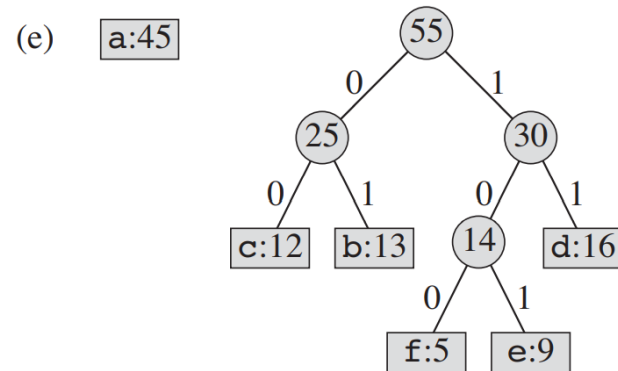
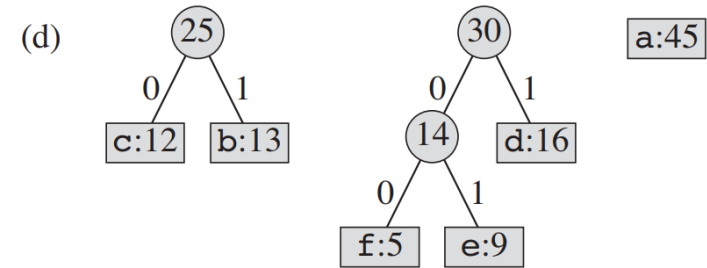
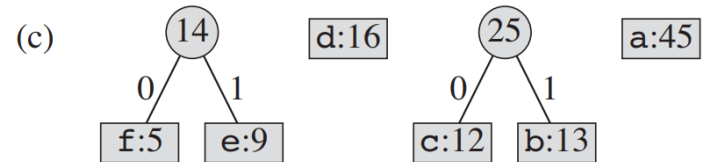
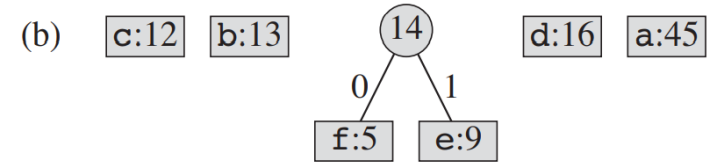
It begins with a set of $|C|$ leaves and performs a sequence of $|C| - 1$ “merging” operations to create the final tree.

- استفاده از min-priority queue یا صف اولویت مینیمم برای بازگرداندن کاراکترهایی با کمترین میزان تکرار

When we merge two objects, the result is a new object whose frequency is the sum of the frequencies of the two objects that were merged.

مثال کدگذاری هافمن

(a) f:5 e:9 c:12 b:13 d:16 a:45



برنامه کد هافمن

HUFFMAN(C)

```
1   $n = |C|$ 
2   $Q = C$  .....  $O(n)$  BUILD-MIN-HEAP
3  for  $i = 1$  to  $n - 1$  .....  $O(n)$ 
4      allocate a new node  $z$  .....  $O(1)$ 
5       $z.left = x = \text{EXTRACT-MIN}(Q)$  .....  $O(\lg n)$ 
6       $z.right = y = \text{EXTRACT-MIN}(Q)$  .....  $O(\lg n)$ 
7       $z.freq = x.freq + y.freq$  .....  $O(1)$ 
8       $\text{INSERT}(Q, z)$  .....  $O(\lg n)$ 
9  return  $\text{EXTRACT-MIN}(Q)$  // return the root of the tree
```

$O(\lg n)$

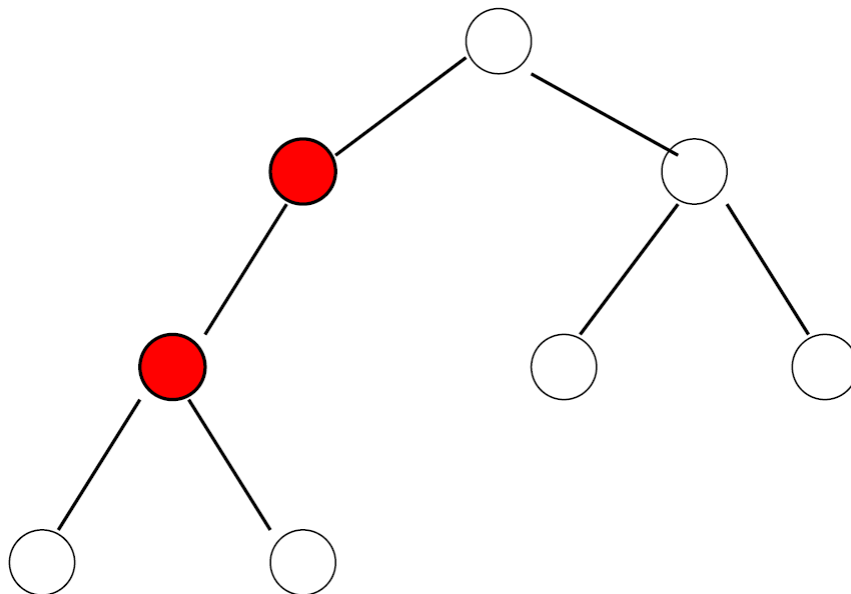
$O(n \lg n)$

اثبات کد هافمن-۱

- مشاهده: درخت کدگذاری بهینه حتما یک درخت پر است!

- به این معنا که هر گره در درخت دقیقا دو فرزند دارد

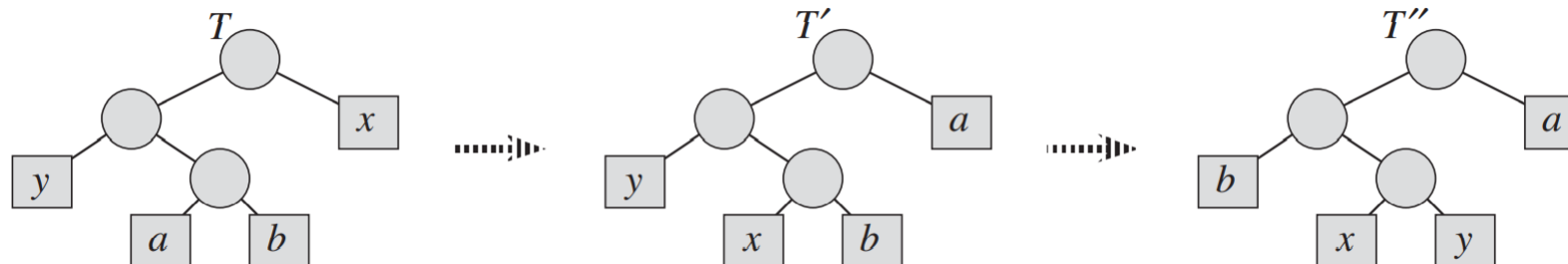
- اثبات:



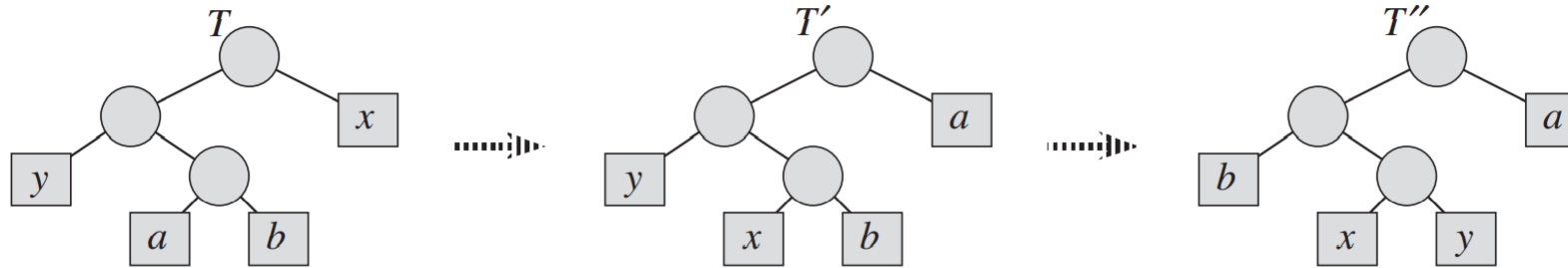
اثبات کد هافمن-۲

• لم: فرض کند X و Y دو کم تکرارترین کاراکتر باشند. پس وجود خواهد داشت کدگذاری بهینه‌ای که در آن این دو کاراکتر دو برگ برادر در پایین‌ترین سطح درخت باشند

• اثبات: فرض کند T درخت برای کدگذاری بهینه باشد و a و b به ترتیب دو برگ برادر در پایین‌ترین سطح درخت آن باشد (چون درخت پر است حتما وجود دارد)
فرض کنید $a.freq \leq b.freq$ و $x.freq \leq y.freq$ باشد. از آنجایی که در فرض X و Y کم تکرارترین کاراکترها هستند پس داریم $x.freq \leq a.freq$ و $y.freq \leq b.freq$ از آنجایی که a و b در عمیق‌ترین سطح قرار دارند پس داریم $d(x) \leq d(a)$ و همچنین $d(y) \leq d(b)$

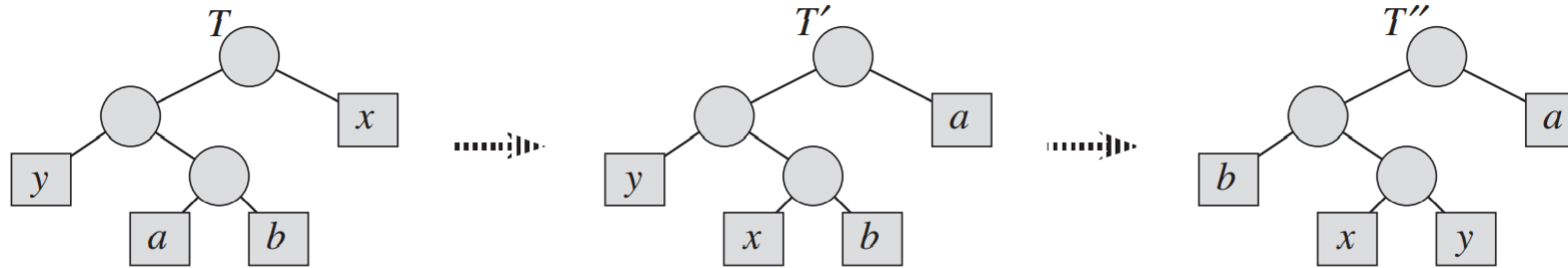


اثبات کد هافمن-۲



$$\begin{aligned}
 & B(T) - B(T') \\
 &= \sum_{c \in C} c.freq \cdot d_T(c) - \sum_{c \in C} c.freq \cdot d_{T'}(c) \\
 &= x.freq \cdot d_T(x) + a.freq \cdot d_T(a) - x.freq \cdot d_{T'}(x) - a.freq \cdot d_{T'}(a) \\
 &= x.freq \cdot d_T(x) + a.freq \cdot d_T(a) - x.freq \cdot d_T(a) - a.freq \cdot d_T(x) \\
 &= (a.freq - x.freq)(d_T(a) - d_T(x)) \\
 &\geq 0,
 \end{aligned}$$

اثبات کد هافمن-۲



because both $a.freq - x.freq$ and $d_T(a) - d_T(x)$ are nonnegative. More specifically, $a.freq - x.freq$ is nonnegative because x is a minimum-frequency leaf, and $d_T(a) - d_T(x)$ is nonnegative because a is a leaf of maximum depth in T . Similarly, exchanging y and b does not increase the cost, and so $B(T') - B(T'')$ is nonnegative. Therefore, $B(T'') \leq B(T)$, and since T is optimal, we have $B(T) \leq B(T'')$, which implies $B(T'') = B(T)$. Thus, T'' is an optimal tree in which x and y appear as sibling leaves of maximum depth, from which the lemma follows. ■

اثبات کد هافمن-۳

• لم: فرض کند T یک درخت دودویی کامل معرف کدگذاری بهینه روی الفای C باشد و $f[c]$ تکرار هر کاراکتر عضو C . هر دو کاراکتر x و y را که در T بصورت برگ برادر ظاهر شدند را در نظر بگیرید و فرض کنید پدر آنها z . در اینصورت، با در نظر گرفتن z بعنوان کارکتری با تکرار $f[z] = f[x] + f[y]$ ، در اینصورت درخت $T' = T - \{x, y\}$ معرف کدگذاری بهینه برای الفای $C' = C - \{x, y\} \cup \{z\}$ است

$$B(T) = B(T') + x.freq + y.freq$$

اثبات کد هافمن-۳

Lemma 16.3

Let C be a given alphabet with frequency $c.freq$ defined for each character $c \in C$. Let x and y be two characters in C with minimum frequency. Let C' be the alphabet C with the characters x and y removed and a new character z added, so that $C' = C - \{x, y\} \cup \{z\}$. Define f for C' as for C , except that $z.freq = x.freq + y.freq$. Let T' be any tree representing an optimal prefix code for the alphabet C' . Then the tree T , obtained from T' by replacing the leaf node for z with an internal node having x and y as children, represents an optimal prefix code for the alphabet C .

اثبات کد هافمن-۳

Proof We first show how to express the cost $B(T)$ of tree T in terms of the cost $B(T')$ of tree T' , by considering the component costs in equation (16.4). For each character $c \in C - \{x, y\}$, we have that $d_T(c) = d_{T'}(c)$, and hence $c.freq \cdot d_T(c) = c.freq \cdot d_{T'}(c)$. Since $d_T(x) = d_T(y) = d_{T'}(z) + 1$, we have

$$\begin{aligned} x.freq \cdot d_T(x) + y.freq \cdot d_T(y) &= (x.freq + y.freq)(d_{T'}(z) + 1) \\ &= z.freq \cdot d_{T'}(z) + (x.freq + y.freq), \end{aligned}$$

from which we conclude that

$$B(T) = B(T') + x.freq + y.freq$$

or, equivalently,

$$B(T') = B(T) - x.freq - y.freq.$$

We now prove the lemma by contradiction. Suppose that T does not represent an optimal prefix code for C . Then there exists an optimal tree T'' such that $B(T'') < B(T)$. Without loss of generality (by Lemma 16.2), T'' has x and y as siblings. Let T''' be the tree T'' with the common parent of x and y replaced by a leaf z with frequency $z.freq = x.freq + y.freq$. Then

$$\begin{aligned} B(T''') &= B(T'') - x.freq - y.freq \\ &< B(T) - x.freq - y.freq \\ &= B(T'), \end{aligned}$$

yielding a contradiction to the assumption that T' represents an optimal prefix code for C' . Thus, T must represent an optimal prefix code for the alphabet C . ■

$$B(T) = \sum_{c \in C} c.freq \cdot d_T(c)$$

اثبات کد هافمن-۴

Theorem 16.4

Procedure HUFFMAN produces an optimal prefix code.

Proof Immediate from Lemmas 16.2 and 16.3.